CS477 Assignment 2 Harris Corner Detection

Computer Vision | 15 October 2026

Hamza Yaqoob (465444)

CV- GRP1

Part A Algorithm and quantities

1. Corner Detection Algorithm

Step	Operation or equation	Input quantities and dimensions	Output quantities and dimensions	Parameters and meaning
1	Load the image and check that reading succeeded.	Image file	Color image M × N × 3, or grayscale M × N; uint8	M and N are pixel counts. Preserve the chosen spatial size.
2	Convert color to grayscale. For BGR input, use 0.114B + 0.587G + 0.299R.	M × N × 3 color image	Grayscale image M × N	Use cvtColor with COLOR_BGR2GRAY; skip for grayscale input.
3	Convert to float32 and divide uint8 intensities by 255.	M × N grayscale image in [0,255]	I: M × N float32 in [0,1]	Intensity convention is fixed before computing responses.
4	Choose derivative kernels. Gaussian derivative kernels each have KD × KD support.	σD, or OpenCV ksize	Two kernels KD × KD, or two Sobel kernels ksize × ksize	KD = 2 ceil(3σD) + 1. OpenCV baseline ksize = 3 pixels.
5	Compute horizontal and vertical first derivatives using the kernels.	I: M × N and two derivative kernels	Ix and Iy: each M × N	σD controls differentiation smoothing. Reflect the image at the boundary.
6	Compute the three pointwise gradient products.	Ix and Iy: M × N each	Ix^2, IxIy, Iy^2: three M × N maps	No extra window is used in this pointwise step.
7	Choose a normalized integration kernel w and integrate the product maps to obtain A, B and C.	Three M × N products; w: KI × KI or b × b	A, B and C: each M × N	Gaussian σI in pixels, KI = 2 ceil(3σI) + 1. Theoretical uniform weights are $1/b^2$; OpenCV uses blockSize = b.

Part A

2. Guassian and OpenCV parameters

$$K_D = 2\lceil 3\sigma_D \rceil + 1$$

$$K_I = 2\lceil 3\sigma_I \rceil + 1$$

Guassian Formulation	OpenCV Implementation
σ_D : derivative scale	ksize : sobel aperture width
$K_D \times K_D$: derivative support	ksize x ksize
σ_I : Integration scale	blockSize : uniform-window width
$K_I \times K_I$: Integration support	blockSize x blockSize

Note: ksize and blockSize are not Gaussian standard deviations.

3. Parameter Units

Parameter	Units	Detector
σ_D, σ_I	Pixels	All three
K_D, K_I	Pixels (kernel width)	All three
ksize, blockSize	Pixel (window width)	OpenCV implement
$k = 0.04$	Dimensionless	Harris only

T_{ST}	Eigenvalue response unit	Shi-Tomasi
T_{DT}	DT response units	Det/Trace
T_H	Harris response unit	Harris
n	Pixels (odd NMS width)	All three
r	Pixels (excluded border)	All three

Note: Threshold ~~differs~~ depends on intensity normalization and the responce definitions

4. Convolution and Derivative Convention

$$(K \times I)(x,y) = \sum_{u,v} K(u,v)\, I(x-u, y-v)$$

+ I_x → intensity increases rightward

+ I_y → intensity increases downward.

OpenCV filter2D() uses correlation. Flip Gaussian derivative kernels when using it to implement mathematical convolution with stated sign convention.

5. Region Classification Using Eigenvalues

Region	Eigenvalue behavior	Interpretation
Flat	$\lambda_1 \approx \lambda_2 \approx 0$	Little intensity variation
Edge	$\lambda_1 \gg \lambda_2 \approx 0$	Strong variation in one direction
Corner	λ_1, λ_2 both large	Strong variation in two direction

H is structure tensor, constructed from integrated first derivative products, not Hessian of second derivative.

6. Thresholding, NMS and Final Coordinates

Thresholding:

$$Q_j(x,y) = \begin{cases} 1, & R_j(x,y) > T_j \\ 0 & \text{otherwise} \end{cases}$$

Non-Maximum Suppression

1. Use an odd $n \times n$ neighborhood such as 3x3.

2. Retain threshold passing pixels whose response equals the maximum in their neighborhood!

3. For connected equal response local maxima plateaus, retain only the pixel with smallest row, then smallest column

4. Use reflected borders (BORDER_REFLECT_101) for filtering and exclude border pixels whose combined derivative, integration and NMS supports extends outside the image.

A conservative Guassian border exclusion width is

$$r = \frac{K_D - 1}{2} + \frac{K_I - 1}{2} + \frac{n-1}{2}$$

For OpenCV Sobel implementation replace K_D, K_I with ksize and blockSize.

- Final Output

$$C_j = \{(x_1, y_1), (x_2, y_2), \ldots, (x_{K_j}, y_{K_j})\}$$

Dimension: $K_j \times 2$ K_j is number of retained corners.

Part B Canny and corner detection

Part B

Similarities

1. Both work on grayscale intensity. This gives a single image for derivative calculation and avoids detecting each color channel seperately.

2. Both use first order derivatives. The local brightness changes provide the evidence for edges and corners.

3. Both use neighborhood filtering and have a noise versus detail tradeoff. Smoothing or large derivative support can reduce noise, but it can also blur features.

4. Both construct a dense strength representation before choosing features A strong intermediate response is evidence for a feature.

5. Both use suppression and thresholding to remove redundant or weak responses. Their threshold depend on intensity scaling and derivative convention.

Part B

Differences

1. Feature and output

Canny seeks intensity boundaries and returns a binary map of thin edge pixels often forming curves.

2. Intermediate quantities

Canny uses gradient magnitude and direction. Harris family detector integrate I_x^2 $I_x I_y$, I_y^2 into 2×2 tensor, then evaluate eigenvalue.

3. Neighborhood aggregation:

Canonical Canny smooths the image before differentiation and evaluates local edge strength.

4. Suppression:

Canny compares gradient magnitude along the gradient direction across an edge. Corner NMS compares a scalar response into 2 dimensional spatial neighborhood such as 3×3 to choose a point.

5. Threshold and connectivity:

Canny uses high and low threshold with hysteresis, keeping weak edge pixels connected to strong ones. Standard haris uses a response threshold and spatial maxima.

Part C OpenCV implementation

Notebook link:

https://colab.research.google.com/drive/1MdMXMiiRzMvJILuzJffT8PFtvepakMQt?usp=sharing

Function or operation	Parameters used	Output and meaning
cv2.imread and cv2.cvtColor	BGR load; COLOR_BGR2GRAY	M × N × 3 uint8 color input becomes M × N grayscale.
cv2.cornerMinEigenVal	image, blockSize, ksize=ksize, borderType	M × N float32 Shi–Tomasi response, the smaller eigenvalue.
cv2.cornerHarris	image, blockSize, ksize, k, borderType	M × N float32 $\det(H) - k\,\mathrm{trace}(H)^2$ response.
cv2.cornerEigenValsAndVecs	image, blockSize, ksize, borderType	M × N × 6 float32: λa, λb, vax, vay, vbx, vby. The eigenvalues are unsorted.
Vectorized DT arithmetic	First two eigenvalue channels; $\varepsilon = 10^{-12}$	M × N float32 λaλb/(λa+λb); zero where the sum is ≤ ε.
cv2.dilate	One 3 × 3 uint8 all-one kernel	M × N local maximum map used with the positive threshold mask.
cv2.connectedComponents	8-connectivity on local maxima	M × N labels; keep the first row-major position per equal-response plateau.
Coordinate extraction and saving	(x,y), zero-based; NumPy and CSV	K × 2 int32 points, dense response .npy, threshold mask .png and overlay .png.

For all three responses, only positive maxima are eligible. If the maximum is non-positive, no corners are returned. Equal-response plateaus are resolved by keeping the pixel with the smallest row, then smallest column.

Part D Images and baseline results

Three responses on the clean images

Baseline settings for every row: ksize = 3, blockSize = 5, α = 0.01, 3 × 3 NMS and a six-pixel border. Harris k = 0.04. T is computed separately for each map. ST means Shi–Tomasi, DT means determinant/trace, and H means classical Harris. Thresholds below are rounded to four significant digits; full precision is in results.csv.

Image	Response	Numerical T	Corners K	Median ms
Synthetic	ST	1.600e-03	18	15.36
Synthetic	DT	9.103e-04	18	35.49

Image	Response	Numerical T	Corners K	Median ms
Synthetic	H	2.828e-04	14	15.17
Building	ST	9.070e-04	1716	15.86
Building	DT	5.367e-04	1680	34.80
Building	H	1.020e-04	542	18.73
Fruits	ST	4.823e-04	1591	17.32
Fruits	DT	3.058e-04	1403	37.13
Fruits	H	3.337e-05	284	16.49

Controlled experiment design

Each sweep changes one factor on the clean synthetic image. The derivative sweep holds blockSize = 5 fixed. The integration sweep holds ksize = 3 fixed. The α and Harris k sweeps use baseline supports. Noise sensitivity compares the baseline clean and noisy image against the same ground truth.

Part D Support experiments

Each detector cell reports numerical T followed by K, the detected count. α = 0.01 and Harris k = 0.04 throughout this page. NMS is 3 × 3 and the excluded border is six pixels.

Derivative support with blockSize 5

ksize	ST threshold and count	DT threshold and count	H threshold and count
3	1.600e-03 K = 18	9.103e-04 K = 18	2.828e-04 K = 14
5	1.281e-02 K = 18	7.919e-03 K = 18	2.207e-02 K = 14
7	1.306e-01 K = 18	8.607e-02 K = 18	2.710e+00 K = 14

Increasing Sobel support from 3 to 5 to 7 did not change the detected counts, F1 or mean localization error on this clean synthetic image. The raw responses and thresholds changed substantially with the discrete derivative support. This result does not show scale invariance or establish which support is best under stronger noise.

Integration support with ksize 3

blockSize	ST threshold and count	DT threshold and count	H threshold and count
3	1.600e-03 K = 18	1.046e-03 K = 18	3.981e-04 K = 14
5	1.600e-03 K = 18	9.103e-04 K = 18	2.828e-04 K = 14
7	1.339e-03 K = 18	7.289e-04 K = 18	1.797e-04 K = 14
9	1.126e-03 K = 18	5.999e-04 K = 18	1.213e-04 K = 14

blockSize	ST and DT F1	ST and DT mean error px	H F1	H mean error px
3	1.000	0.079	0.875	0.101
5	1.000	1.493	0.875	1.515
7	0.944	2.828	0.813	2.828
9	0.000	Not applicable	0.000	Not applicable

Counts stayed constant while localization worsened. At blockSize = 9, all selected maxima were more than three pixels from their associated ground-truth vertices, so the strict matching rule gave zero matches. The detections still appear near the correct shapes. This is a localization failure under the stated tolerance, not evidence that the response map contains no corner signal.

Part D Threshold Harris k and noise

Relative threshold with baseline supports

α	ST threshold and count	DT threshold and count	H threshold and count
0.01	1.600e-03 K = 18	9.103e-04 K = 18	2.828e-04 K = 14
0.05	8.000e-03 K = 14	4.552e-03 K = 14	1.414e-03 K = 14

α	ST threshold and count	DT threshold and count	H threshold and count
0.1	1.600e-02 K = 14	9.103e-03 K = 14	2.828e-03 K = 14

ST and DT lost the four low-contrast corners when α increased to 0.05. Their F1 fell from 1.000 to 0.875 and stayed there at 0.10. Harris already missed those four at α = 0.01, so its count and F1 did not change in this sweep. A larger relative threshold removes weaker features as well as possible noise.

Harris sensitivity with baseline supports

k	α	Numerical T	Corners K	F1
0.02	0.01	3.104e-04	14	0.875
0.04	0.01	2.828e-04	14	0.875
0.06	0.01	2.552e-04	14	0.875
0.08	0.01	2.277e-04	14	0.875

All tested k values retained the same 14 corners with the same localization. Increasing k subtracts a larger trace-squared penalty and theoretically rejects highly unequal eigenvalues more strongly. Here the change in k did not improve recall, and the relative threshold changed along with the maximum response.

Noise at the baseline settings

Response	Clean T	Noisy T	Clean K	Noisy K
ST	1.600e-03	1.625e-03	18	18
DT	9.103e-04	9.234e-04	18	23
H	2.828e-04	2.927e-04	14	14

Only DT gained extra detections in this noise test. Its five unmatched points were (230,42), (108,120), (110,120), (337,353) and (337,364), all near straight edges. ST kept all 18 true corners without adding false positives. Harris kept 14 and still missed the weak rectangle. These findings apply to this image, noise level and seed.

Using the same α makes the experiment reproducible, but it does not make the three detectors equally selective. Their response distributions differ, and Harris has fourth-power intensity scaling

while ST and DT have second-power scaling. A single very strong corner also sets the relative threshold for the entire image.

Part D Synthetic overlays



Figure 1. Clean synthetic image above and noisy copy below. Red circles mark final detections. Every panel states its response, numerical threshold and common support settings; d is ksize and b is blockSize. Coordinates are zero-based. All images use the same size and intensity display range [0,1].

The weak rectangle is at the lower left. ST and DT detect its corners on the clean image, while Harris does not. On the noisy image, the extra DT marks along edges explain its lower precision. The large flat areas do not produce retained baseline detections.

Ground truth consists of (40,40), (120,40), (120,120), (40,120), (180,40), (270,40), (270,115), (180,115), (40,190), (120,190), (120,275), (40,275), (190,190), (300,190), (300,230), (240,230), (240,300) and (190,300). The straight edge at x = 340 has no interior corner.

Part D Natural image overlays

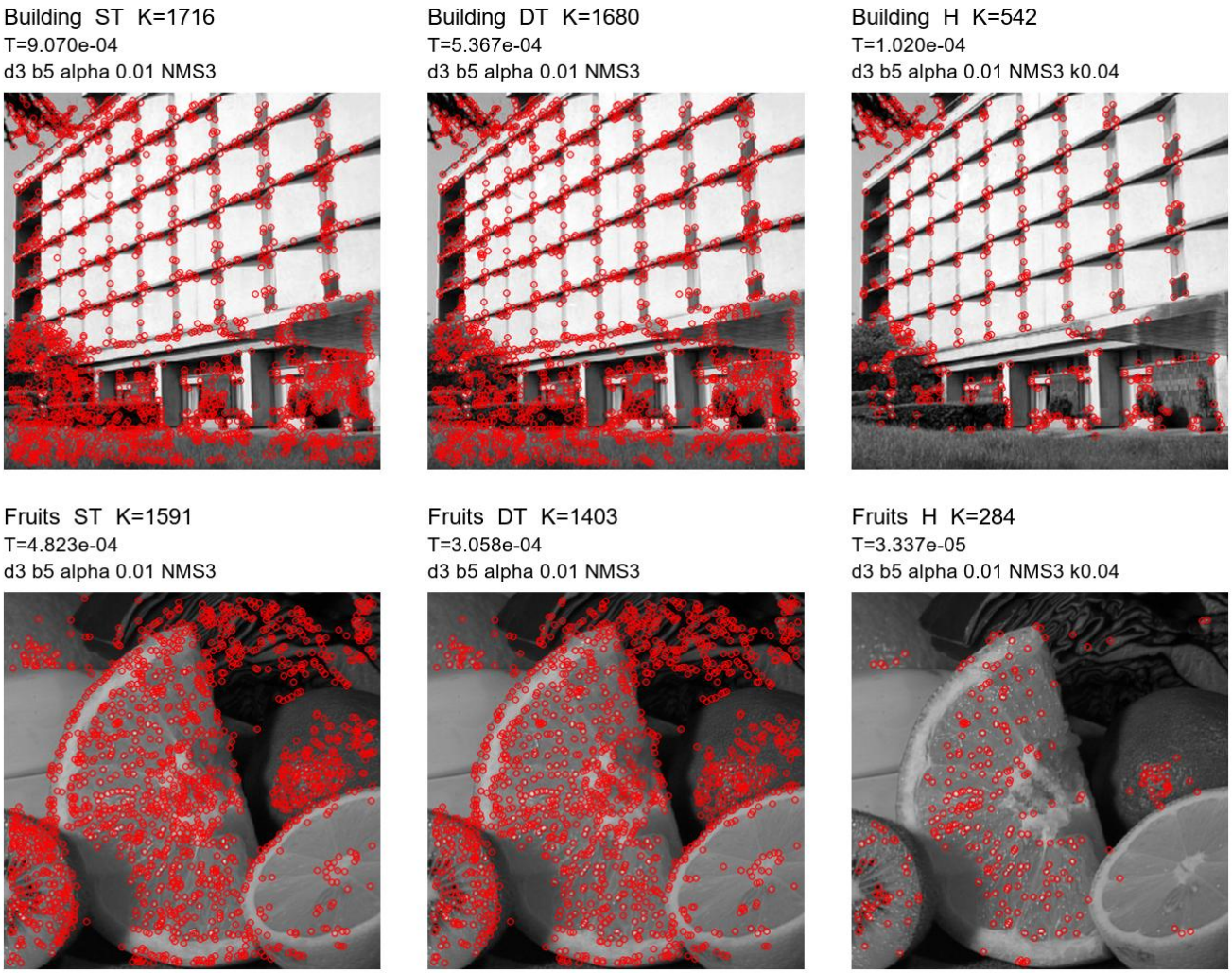


Figure 2. Building above and fruits below, with the same display scale and red detection markers for each response. All use ksize = 3, blockSize = 5, α = 0.01, NMS = 3 × 3 and a six-pixel border. Harris uses k = 0.04. Image sources are OpenCV samples. [2,3]

The building results contain useful window and facade junctions along with detections in foliage and surface texture. ST and DT give much denser coverage. Harris gives a sparser set that emphasizes stronger junctions, though some texture remains.

In the fruit image, ST and DT produce many points on textured surfaces and clutter, rather than just object boundaries. Harris gives fewer points. Natural-image ground truth was not annotated, so these overlays support a qualitative comparison; their counts do not establish precision or prove that the sparsest detector is the most accurate.

PART E — COMPARE AND INTERPRET PERFORMANCE

Image	Resp	K	TP	FP	FN	Precision	Recall	F1	Error px
Synthetic	ST	18	18	0	0	1.000	1.000	1.000	1.493
Synthetic	DT	18	18	0	0	1.000	1.000	1.000	1.493

Image	Resp	K	TP	FP	FN	Precision	Recall	F1	Error px
Synthetic	H	14	14	0	4	1.000	0.778	0.875	1.515
Noisy	ST	18	18	0	0	1.000	1.000	1.000	1.493
Noisy	DT	23	18	5	0	0.783	1.000	0.878	1.493
Noisy	H	14	14	0	4	1.000	0.778	0.875	1.515

ST and DT tie on the clean image with F1 = 1.000 and mean error 1.493 pixels. In noise, DT recall stays at 1.000 but its precision falls to 0.783 because of five edge detections. Harris precision is 1.000 for both images, but recall is 0.778 because it misses four low-contrast corners. Counting detections alone would hide these differences.

Measured runtime

Times are for the full detection call: OpenCV response calculation, thresholding, dilation, border exclusion, plateau labeling and coordinate extraction. Each row uses one warm-up call and the median of five timed calls on the same CPU and 384 × 384 image. Loading, generation, saving, plotting, evaluation and report creation are excluded. Each branch recomputes its gradients and tensor through its own library call; no shared tensor is reused.

Image	ST ms	DT ms	H ms
Synthetic	15.36	35.49	15.17
Building	15.86	34.80	18.73
Fruits	17.32	37.13	16.49
Noisy	15.02	35.28	15.19

DT was slower in every baseline comparison. Its branch requests both eigenvalues and eigenvectors before computing the ratio, which likely contributes to the extra time. ST and Harris were closer in runtime. These CPU timings can vary with system load; they describe this implementation and machine rather than a universal ranking.

Part E Interpretation and recommendations

What the parameters change

Derivative support controls how much neighboring image structure contributes to each derivative. Larger support generally smooths high-frequency noise but can weaken fine structure and move

responses near small features. The clean support sweep showed unchanged matching results, so a noise advantage was not measured here.

Integration support combines derivative evidence over a wider region. This can stabilize an estimate, but it can also mix nearby corners or shift a maximum away from the physical vertex. The measured blockSize sweep showed this localization cost clearly: ST and DT mean error rose from 0.079 pixels at blockSize = 3 to 1.493 at 5 and 2.828 at 7 for matched points. At 9, no points met the three-pixel tolerance. The shapes were deliberately well separated, so nearby-corner merging was not directly tested.

Threshold α controls how weak a response can be relative to the strongest response in its own map. Increasing it reduced weak-corner recall in ST and DT. The Harris k controls the penalty for a large trace: stronger directional imbalance is penalized as k increases. On this synthetic image, changing k from 0.02 to 0.08 did not change the retained corners.

Gaussian and uniform weighting

Uniform integration gives the same weight to every derivative product inside its window and zero weight outside. Gaussian integration gradually reduces the influence of products farther from the center. It can reduce abrupt window-boundary effects, but σI must still balance noise averaging against localization and corner separation. These are conceptual expectations; this study measured uniform integration only.

$$\text{Variance per axis of a uniform width } b = \frac{b^2 - 1}{12}$$

A comparison with a Gaussian of equal per-axis variance would use σ = square root of (b^2 − 1)/12, while also reporting its truncation support. Equal variance does not make the filters identical. Neither this relation nor the support-size sweep demonstrates scale invariance.

Settings supported by the evidence

For clean synthetic corners, ST with ksize = 3, blockSize = 3 and α = 0.01 was the best measured choice when both recall and localization matter. DT tied its F1 and localization but took longer. ST at the required baseline also handled the measured noisy image best: it retained all 18 corners with no false positives. The smaller block was not tested on the noisy copy, so its noise performance remains unmeasured.

For the building image, baseline Harris is a reasonable choice when a compact set of strong junctions is useful; its 542 detections were easier to inspect than the 1716 ST and 1680 DT points. For dense tracking coverage, ST is also useful. For fruits, baseline Harris reduced the dense texture responses to 284 points, compared with 1591 ST and 1403 DT points. These natural-image preferences are based on the displayed overlays and application needs. Only baseline parameters were tested on these photographs, so they are starting choices rather than demonstrated optima.

References and reproducibility

[1] OpenCV. Feature Detection API, version 4.13.0. Documentation for cornerMinEigenVal, cornerHarris, cornerEigenValsAndVecs and Canny. Accessed 15 October 2026. https://docs.opencv.org/4.13.0/dd/d1a/group__imgproc__feature.html

[2] OpenCV sample image building.jpg, release 4.13.0. Source photograph used for the structured image experiment. https://github.com/opencv/opencv/blob/4.13.0/samples/data/building.jpg

[3] OpenCV sample image fruits.jpg, release 4.13.0. Source photograph used for the cluttered image experiment. https://github.com/opencv/opencv/blob/4.13.0/samples/data/fruits.jpg

Appendix Complete working code

Save the Python listing as experiment.py and the settings as config.json in the same folder. Keep the supplied images folder beside them. Install the dependencies below, then run python experiment.py --config config.json --output results.

Python script

```
import argparse
import csv
import hashlib
import json
import platform
import time
from pathlib import Path

import cv2
import numpy as np


def make_images(folder, seed, size):
    folder.mkdir(parents=True, exist_ok=True)
    if size != 384:
        raise ValueError("Use 384 for the supplied synthetic ground truth")
    image = np.full((size, size), 0.1, np.float32)
    vertices = []
    for x1, y1, x2, y2, value in [
        (40, 40, 120, 120, 0.9),
        (180, 40, 270, 115, 0.55),
        (40, 190, 120, 275, 0.25),
    ]:
        cv2.rectangle(image, (x1, y1), (x2, y2), value, -1)
        vertices.extend([(x1, y1), (x2, y1), (x2, y2), (x1, y2)])
    polygon = np.array([(190, 190), (300, 190), (300, 230),
                        (240, 230), (240, 300), (190, 300)], np.int32)
    cv2.fillPoly(image, [polygon], 0.85)
    vertices.extend(polygon.tolist())
    image[:, 340:] = 0.7
    noise = np.random.default_rng(seed).normal(0, 0.02, image.shape)
    images = {"synthetic": image,
              "noisy": np.clip(image + noise, 0, 1).astype(np.float32)}
    for name in ("building", "fruits"):
        original = cv2.imread(str(folder / f"{name}_original.jpg"))
        if original is None:
            raise FileNotFoundError(folder / f"{name}_original.jpg")
        gray = cv2.cvtColor(original, cv2.COLOR_BGR2GRAY)
        h, w = gray.shape
        side = min(h, w)
        y0, x0 = (h - side) // 2, (w - side) // 2
        images[name] = cv2.resize(gray[y0:y0+side, x0:x0+side],
```

```
                                    (size, size), interpolation=cv2.INTER_AREA).astype(np.float32) /
255
    for name, img in images.items():
        np.save(folder / f"{name}.npy", img)
        cv2.imwrite(str(folder / f"{name}.png"), np.rint(img * 255).astype(np.uint8))
    truth = np.asarray(vertices, np.float64).reshape(-1, 2)
    np.savetxt(folder / "ground_truth.csv", truth, delimiter=",", header="x,y", comments="",
fmt="%.1f")
    return images, truth


def response_map(image, detector, ksize, block_size, k, epsilon):
    border = cv2.BORDER_REFLECT_101
    if detector == "ST":
        return cv2.cornerMinEigenVal(image, block_size, ksize=ksize, borderType=border)
    if detector == "H":
        return cv2.cornerHarris(image, block_size, ksize, k, borderType=border)
    eig = cv2.cornerEigenValsAndVecs(image, block_size, ksize, borderType=border)
    a, b = eig[..., 0], eig[..., 1]
    trace = a + b
    out = np.zeros(image.shape, np.float32)
    np.divide(a * b, trace, out=out, where=trace > epsilon)
    return out


def select_corners(response, alpha, border, nms):
    maximum = float(response.max())
    threshold = alpha * maximum
    mask = (response > threshold) & (response > 0) if maximum > 0 else np.zeros(response.shape,
bool)
    local_max = cv2.dilate(response, np.ones((nms, nms), np.uint8))
    candidates = mask & (response == local_max)
    candidates[:border] = False
    candidates[-border:] = False
    candidates[:, :border] = False
    candidates[:, -border:] = False
    count, labels = cv2.connectedComponents(candidates.astype(np.uint8), connectivity=8)
    first = np.full(count, response.size, np.int64)
    flat = np.flatnonzero(candidates)
    np.minimum.at(first, labels.ravel()[flat], flat)
    chosen = first[1:]
    y, x = np.divmod(chosen, response.shape[1])
    points = np.column_stack((x, y)).astype(np.int32).reshape(-1, 2)
    return points, mask, threshold


def detect(image, detector, ksize, block_size, k, alpha, epsilon, border, nms):
    response = response_map(image, detector, ksize, block_size, k, epsilon)
    points, mask, threshold = select_corners(response, alpha, border, nms)
    return response, points, mask, threshold


def evaluate(points, truth, border, size, tolerance):
    valid = np.all((truth >= border) & (truth < size - border), axis=1)
    truth = truth[valid]
    distance = np.linalg.norm(points[:, None, :] - truth[None, :, :], axis=2)
    neighbors = [sorted(np.flatnonzero(row <= tolerance).tolist(), key=lambda j: (row[j], j))
                 for row in distance]
    assigned = {}

    def augment(i, seen):
        for j in neighbors[i]:
            if j in seen:
                continue
            seen.add(j)
```

```
            if j not in assigned or augment(assigned[j], seen):
                assigned[j] = i
                return True
        return False

    for i in range(len(points)):
        augment(i, set())
    tp = len(assigned)
    fp, fn = len(points) - tp, len(truth) - tp
    precision = tp / len(points) if len(points) else 0.0
    recall = tp / len(truth) if len(truth) else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    error = float(np.mean([distance[i, j] for j, i in assigned.items()])) if tp else None
    return dict(tp=tp, fp=fp, fn=fn, precision=precision, recall=recall, f1=f1,
                mean_error_px=error, gt_count=len(truth))


def configurations(settings):
    base = settings["baseline"]
    runs = []

    def add(group, image, detector, **changes):
        cfg = dict(base)
        cfg.update(changes)
        cfg.update(group=group, image=image, detector=detector)
        runs.append(cfg)

    for image in ("synthetic", "building", "fruits"):
        for detector in ("ST", "DT", "H"):
            add("baseline", image, detector)
    for key, group in (("ksize", "derivative"), ("block_size", "integration"), ("alpha",
"threshold")):
        for value in settings["sweeps"][key]:
            if value == base[key]:
                continue
            for detector in ("ST", "DT", "H"):
                add(group, "synthetic", detector, **{key: value})
    for value in settings["sweeps"]["k"]:
        if value != base["k"]:
            add("harris_k", "synthetic", "H", k=value)
    for detector in ("ST", "DT", "H"):
        add("noise", "noisy", detector)
    return runs


def run(config_path, output):
    settings = json.loads(config_path.read_text())
    cv2.setNumThreads(1)
    cv2.ocl.setUseOpenCL(False)
    root = config_path.parent
    images, truth = make_images(root / "images", settings["seed"], settings["size"])
    output.mkdir(parents=True, exist_ok=True)
    rows = []
    for cfg in configurations(settings):
        image = images[cfg["image"]]
        args = (image, cfg["detector"], cfg["ksize"], cfg["block_size"], cfg["k"],
                cfg["alpha"], settings["epsilon"], settings["border"], settings["nms"])
        detect(*args)
        samples = []
        for _ in range(5):
            start = time.perf_counter_ns()
            result = detect(*args)
            samples.append((time.perf_counter_ns() - start) / 1e6)
        response, points, mask, threshold = result
        name = (f"{cfg['image']}_{cfg['detector']}_d{cfg['ksize']}_b{cfg['block_size']}"
                f"_a{cfg['alpha']:.2f}_k{cfg['k']:.2f}")
```

```
        folder = output / name
        folder.mkdir(exist_ok=True)
        np.save(folder / "response.npy", response)
        cv2.imwrite(str(folder / "threshold_mask.png"), mask.astype(np.uint8) * 255)
        np.save(folder / "corners.npy", points)
        np.savetxt(folder / "corners.csv", points, delimiter=",", header="x,y", comments="",
fmt="%d")
        overlay = cv2.cvtColor(np.rint(image * 255).astype(np.uint8), cv2.COLOR_GRAY2BGR)
        for x, y in points:
            cv2.circle(overlay, (int(x), int(y)), 3, (0, 0, 255), 1, cv2.LINE_AA)
        cv2.imwrite(str(folder / "overlay.png"), overlay)
        row = dict(cfg, run_id=name, threshold=threshold, maximum=float(response.max()),
                   count=len(points), runtime_ms=float(np.median(samples)),
                   nms=settings["nms"], border=settings["border"], epsilon=settings["epsilon"])
        if cfg["image"] in ("synthetic", "noisy"):
            row.update(evaluate(points, truth, settings["border"], settings["size"],
settings["tolerance"]))
        rows.append(row)
    fields = list(dict.fromkeys(key for row in rows for key in row))
    with (output / "results.csv").open("w", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)
    (output / "results.json").write_text(json.dumps(rows, indent=2))
    metadata = dict(python=platform.python_version(), opencv=cv2.__version__, numpy=np.__version__,
                    platform=platform.platform(), processor=platform.processor(), threads=1,
                    timing="One warm-up then median of five runs; response and NMS recomputed; no
I/O, plotting or evaluation",
                    seed=settings["seed"], noise_std=0.02,
                    hashes={p.name: hashlib.sha256(p.read_bytes()).hexdigest()
                            for p in (root / "images").glob("*_original.jpg")})
    (output / "environment.json").write_text(json.dumps(metadata, indent=2))
    print(json.dumps(dict(runs=len(rows), output=str(output), environment=metadata), indent=2))


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--config", type=Path, default=Path(__file__).with_name("config.json"))
    parser.add_argument("--output", type=Path, default=Path(__file__).with_name("results"))
    args = parser.parse_args()
    run(args.config.resolve(), args.output.resolve())
```

Configuration file

```
{
  "baseline": {"ksize": 3, "block_size": 5, "k": 0.04, "alpha": 0.01},
  "sweeps": {"ksize": [3, 5, 7], "block_size": [3, 5, 7, 9], "alpha": [0.01, 0.05, 0.10], "k":
[0.02, 0.04, 0.06, 0.08]},
  "seed": 47702,
  "size": 384,
  "epsilon": 1e-12,
  "border": 6,
  "nms": 3,
  "tolerance": 3.0
}
```

Dependencies

```
numpy==2.4.2
opencv-python-headless==4.13.0.92
```

Run commands

```
python -m pip install -r requirements.txt
```

```
python experiment.py --config config.json --output results
```